

Q-1: K-th Occurrence in Range

You are given a sequence A of N integers.

The task is to answer Q queries on the given sequence. For each query, you will be given four space-separated integers L, R, P, K .

Return the index of the K th occurrence of P in the range $[L, R]$ (inclusive). If no such index exists, return -1 .

Example Test Cases:

TEST CASE 1

Input:

10 3

1 1 2 1 2 3 1 2 3 4

1 10 2 3

1 5 2 3

5 9 3 2

Output:

8

-1

9

Explanation:

- In the first query, the 3rd occurrence of 2 in range $[1, 10]$ is at index 8.
- In the second query, there are only two occurrences of 2 in range $[1, 5]$, so the answer is -1 .
- In the third query, the 2nd occurrence of 3 in range $[5, 9]$ is at index 9.

Constraints:

- $2 \leq N, Q \leq 10^5$
- $1 \leq A_i \leq 10^6$
- $1 \leq L < R \leq N$
- $1 \leq P \leq 10^6$

- $1 \leq K \leq N$

Expected Time Complexity: $\Theta(N \log N + Q \log N)$

Input Format:

1. The first line contains two space-separated integers N and Q — the size of the array and the number of queries.
2. The second line contains N space-separated integers that make up the array A (1-based indexing).
3. The next input is a single vector of queries provided in the form of `vector<vector<int>>` q , where each inner vector contains four integers L , R , P , and K .

Output Format:

Return a `vector<int>` of size equal to the number of queries. Each element in the vector corresponds to the answer for a query, i.e., the index of the K th occurrence of P in the range $[L, R]$ (inclusive), or -1 if no such occurrence exists.

Q-2: Divide and Summarize

Given an array A of length N , answer Q queries. For each query, determine if it is possible to obtain an **array with a sum** equal to S_i by performing a sequence of slicing operations. The first slice has to be on the input array A .

A slicing operation is defined as follows:

1. Compute $\text{mid} = \lfloor (\max(A) + \min(A)) / 2 \rfloor$.
2. Partition the array into two arrays:
 - **Left array:** Contains all elements with value $\leq \text{mid}$.
 - **Right array:** Contains all elements with value $> \text{mid}$.
3. Choose one of the two arrays to retain, discarding the other.

You must determine the results of all Q queries.

Example Test Cases:**TEST CASE 1**

Input:

5 5

1 2 3 4 5

1 8 9 12 6

Output:

Yes

No

Yes

No

Yes

Explanation:

The given array is [1, 2, 3, 4, 5].

If we perform the slicing operation once, it results in two arrays: [1,2,3] and [4,5].

On the second slice:

[1,2,3] splits into [1,2] and [3].

[4,5] splits into [4] and [5].

On the third slice, [1,2] splits into [1] and [2].

We cannot perform further slicing, so the possible generated subarrays are:

[1], [2], [3], [4], [5], [1,2], [4,5], [1,2,3], [1,2,3,4,5].

All the possible sums are: 1,2,3,4,5,6,9,15.

Thus, queries with these sums will return Yes, and others will return No.

TEST CASE 2

Input:

5 5
3 1 3 1 3
1 2 3 9 11

Output:

No
Yes
No
Yes
Yes

Explanation:

The given array is $[3, 1, 3, 1, 3]$.

If we perform the slicing operation once, it results in two arrays: $[1, 1]$ and $[3, 3, 3]$.

Since no further slicing is possible, the possible sums are 2, 9, 11.

Thus, only queries with these sums will return **Yes**, and others will return **No**.

Expected Time Complexity: $\Theta(n \cdot \log(\max(A)) + Q \cdot \log n)$

Input Format:

1. The first line contains two space-separated integers N and Q — the size of the array and the number of queries.
2. The second line contains N space-separated integers representing the array A .
3. The next input is a single vector containing Q integers S_i , representing the target sum for each query.

Output Format:

For each test case, return a `vector<bool>` of size Q . Each element should be "true" if the corresponding query is satisfied and "false" otherwise.

Instructions:

You have been provided with a template file containing a `main` function that takes the input and calls the `solve` function for both questions. The first line of input for each question contains T — the number of test cases. The `solve` function is called separately for each test case. Your task is to implement the `solve` function for each question.

The `solve` function must:

- For $Q - 1$ (K-th index in query): Return a vector of size Q , containing the index of the K -th occurrence of P in the range $[L, R]$. If no such index exists, return -1 .
- For $Q - 2$ (Divide and Summarize): Return a vector of size Q , containing "true" if the sum S_i can be obtained through a sequence of valid partition operations, otherwise return "false".

Note: Refer to the `STL-syntax.cpp` file provided for the syntax of STL containers. (Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)